

consumes around 67GB RAM to load, whereas training a 200-billion-parameter model would demand thousands of high-end GPUs running nonstop for several months, with enormous power, cooling, and engineering overheads.

RAG unlocks efficiency by querying external, up-to-date sources for real-time or incremental updates, eliminating the need for full retraining. In enterprise settings, models with 3-10 billion parameters, fine-tuned to domain-specific knowledge, achieve rapid deployment, lower infrastructure costs, and superior task-specific accuracy, without the operational overhead of hyperscale AI.

How parameters drive hardware needs: RAM, CPU, GPU

Model parameters dictate RAM requirements directly:

- 3 billion parameters → ~ 30GB RAM load
- 7 billion parameters → ~ 67GB RAM load
- 13 billion parameters → well over 120GB RAM load

Enterprises no longer require a single monolithic generative AI model for every function. Instead, they can deploy multiple, domain-specific models, each fine-tuned for a team or department. For instance, a design team could leverage a model optimised for engineering workflows, while HR or finance operate separate models tailored to their processes. Open source platforms such as LLaMA, Mistral, Falcon, Bloom, or Stable Diffusion can be fine-tuned on industry-specific datasets, delivering higher relevance, faster performance, and lower operational costs.

Model size directly informs hardware selection: lightweight models (≤10 billion parameters) run efficiently on AI-enabled laptops; medium models (~ 20 billion parameters) are suited for single-socket server CPUs; models with 100 billion parameters demand multi-socket configurations. Full training

Modern server-grade CPUs, such as Intel's 4th-Generation Sapphire Rapids or above, embed AI accelerators like Advanced Matrix Extensions (AMX). These offload matrix multiplications are crucial to AI training and inference, freeing up CPU cores and enhancing PyTorch performance for models such as LLaMA.

of trillion-parameter models remains the preserve of large enterprises.

Software optimisation for existing hardware

Optimising generative AI performance is not just about choosing the right model; it is also about knowing how to extract maximum efficiency from the underlying hardware. A simple example illustrates the point.

On an x86 architecture server CPU with 40 cores, parsing a 2.4GB CSV file using Pandas, an open source Python library for working with datasets, took approximately 340 seconds. With just a single line change in existing code to use the optimised version of Pandas called Modin, with 'import modin.pandas' instead of pandas, the parse time dropped to 31 seconds, delivering a 10x speed-up without altering the rest of the code. This small optimisation leverages hardware parallelism more effectively, dramatically reducing processing time.

The takeaway for AI engineers is not to focus solely on the model or framework. Investigate what hardware the client is using, study its acceleration features, and apply relevant optimisations, whether through libraries such as Modin, IPEx, hardware-aware compilers, or vendor-specific AI toolkits.

Open ecosystem standards: OPEA and beyond

Exploring generative AI often faces a steep hardware optimisation curve, especially when tuning models for specific industry needs. One way to simplify the process is through the Open Platform for Enterprise

AI (OPEA). Backed by the Linux Foundation, OPEA is a collaboration among major hardware and software players, including Intel, AMD, SAP, Infosys, Wipro, Docker, and others.

The initiative delivers open source architecture blueprints, reference implementations, and benchmarking tools for standard GenAI models. More than 30 enterprise use cases, from banking chatbots to manufacturing defect detection, are already covered. (By early 2026, some estimates suggested that, when broken down by specific tasks, there were over 600 examples of how organisations were using generative AI.) Crucially, these resources come pre-optimised for a range of hardware platforms, meaning enterprises can avoid the trial-and-error of manual tuning.

By leveraging OPEA's open source repository at opea.dev, teams can reduce both development time and infrastructure costs while maintaining performance. The approach complements techniques such as fine-tuning (for injecting large volumes of domain-specific historical data) and RAG (for feeding real-time external data into the model). Together, these methods enable smaller, cost-efficient AI models to achieve enterprise-grade accuracy, without the operational burden of scaling up to trillion-parameter behemoths.

Optimising enterprise GenAI means structuring workloads so that each task is handled by a model tailored to its domain. By aligning model design with specific operational goals and integrating RAG for real-time relevance, organisations create leaner, faster, and more accurate systems, maximising both hardware efficiency and the value of their data without overextending resources. **EFY**

This article is based on the session titled 'Optimising Generative AI for Hardware' delivered by Anish Kumar, AI Software Engineering Manager from Intel Technology India Pvt Ltd, at AI DevCon 25 in Bengaluru. The session was transcribed and curated by Janarthana Krishna Venkatesan, Journalist at EFY, until recently.